

# Instructor Handout 3: IEEE 754 Floating-Point Representation and Arithmetic

Auqib Hamid Lone

*Department of Computer Science & Engineering, GCET Ganderbal*

August 8, 2026

**Instructor Copy — Not for Student Distribution**

## 3.1 Before You Teach This Lecture

- **Patterson & Hennessy, *Computer Organization and Design*, ARM Edition** — Sec. 3.5 “Floating Point,” p.205. Page-cited — index built 2026-08-07. This is your precise, page-citable anchor for the format, the add/multiply algorithms, and rounding.
- **Hamacher, *Computer Organization*, Ch. 9** — *not page-indexed*. Cite at chapter level for the general floating-point treatment.
- **Do the thread arithmetic before class:** encode  $10.75 (= 1.01011_2 \times 2^3)$  as `0x412C0000`, and trace the qualitative add of  $10.75 + 0.1$ . Have the field breakdown written out so the board numbers are right the first time.
- **Plan the two worked decodings** (the Socratic checks): `0x3F800000` = 1.0, and the GATE-style  $2.27 \times 10^{-1}$  decode. Both are good “do it live” problems.

## 3.2 Scientific Notation

Open by asking how to write  $10^{-30}$  compactly — the answer (scientific notation) already contains the whole idea: sign, mantissa, exponent. The point to extract: a fixed point would waste hundreds of bits placing that “1”; floating point stores only the mantissa’s precision, and the exponent carries the range.

**Definition (Floating-Point Value).** *A floating-point number represents  $(-1)^S \times M \times r^E$ , where  $S$  is the sign bit,  $M$  the mantissa,  $E$  the exponent, and  $r$  the base (2 in hardware). In base 2, a normalized value is  $1.xxx_2 \times 2^E$ .*

State the two knobs explicitly: **range** comes from the exponent, **precision** from the mantissa — independent, which is the entire point of the format. (P&H, Sec. 3.5, p.205; Hamacher Ch. 9) [2, 1]

**Example 3.2.1** (Binary scientific notation).  $10.75 = 8 + 2 + 0.5 + 0.25 = 1010.11_2 = 1.01011_2 \times 2^3$ : sign 0, mantissa 01011, exponent 3. This is the running thread’s starting point.

**How to present it:** derive the binary expansion on the board from place values ( $8 + 2 + 0.5 + 0.25$ ), then normalize by moving the point until the leading digit is 1. The “leading 1 is redundant” observation — that every nonzero number in  $[1, 2)$  starts with 1 — is the seed of the hidden-bit trick.

**Teaching note:** The most common student error is normalizing to  $0.101011 \times 2^4$  instead of  $1.01011 \times 2^3$  — any representation with the leading 1 in the ones place. Make the uniqueness point explicitly: normalization is what gives *one* canonical form. The hidden-bit trick then follows: since the leading 1 is always there, don’t store it.

### 3.3 IEEE 754 Single Precision

Introduce the format as “scientific notation, compressed into 32 bits.” Show the layout (Figure 3.1).

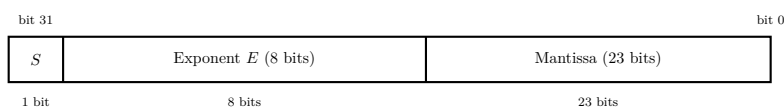


Figure 3.1: IEEE 754 single precision: 1 sign + 8 exponent + 23 mantissa, value =  $(-1)^S \times (1.M)_2 \times 2^{E-127}$ .

**How to present it:** draw the 32-bit word as one long rectangle, then the two divider lines, then label the three fields. Fill in the field widths *after* the labels — the class will wonder why the mantissa gets the most bits, which is itself a useful question (precision dominates for most scientific values).

Then the value formula and the bias:

$$(-1)^S \times (1.M)_2 \times 2^{E-\text{bias}}, \quad \text{bias} = 127.$$

**Definition (Biased Exponent).** *The stored exponent is  $E_{\text{stored}} = E_{\text{true}} + 127$  (single precision). The bias makes the stored field unsigned and correctly ordered — all-zeros means smallest, all-ones largest — so unsigned comparison orders floats. The extreme all-0 and all-1 patterns are reserved for special values.*

**Teaching note:** Students ask why not just use two’s complement for the exponent. The answer is *comparison*: if the exponent were two’s complement, comparing two floats would require sign-aware comparison logic; with a bias, plain unsigned comparison of the bit patterns gives the right order. State this as the design reason, not a convention.

**Example 3.3.1** (Encoding 10.75).  $10.75 = 1.01011_2 \times 2^3$ : sign 0; mantissa field 01011 followed by zeros to 23 bits; exponent  $3 + 127 = 130 = 1000\,0010_2$ . The 32-bit word is 0 10000010 01011000..., i.e. `0x412C0000`. Check:  $1.01011_2 \times 2^{130-127} = 1010.11_2 = 10.75$ .

**A question worth asking live**, right after the encoding: “what is the single most common bit pattern in the whole format — i.e. decode the exponent 01111111 with mantissa 0?” (1.0; the bias is chosen so 1.0 is the anchor pattern.)

**Teaching note:** The encoding example is where everything could go wrong if rushed. Go field by field on the board: sign first, then exponent (compute  $3 + 127$  aloud), then pad the mantissa to 23 bits. Do the check expansion *afterwards* — it validates the whole procedure and catches any arithmetic slip in front of the class.

### 3.4 Normalization, Special Values, and Rounding

Cover the reserved exponent patterns as the format’s “extreme-case behavior.” State each pattern and its meaning (P&H, Sec. 3.5, p.205) [2]: zero (exp 0, mant 0), denormal (exp 0, mant  $\neq 0$ , leading 1 not assumed), infinity (exp all 1, mant 0), NaN (exp all 1, mant  $\neq 0$ ).

**Example 3.4.1** (The smallest and largest normalized single-precision values). *Smallest normalized: exponent 1, mantissa 0  $\Rightarrow 1.0 \times 2^{-126} \approx 1.18 \times 10^{-38}$ . Largest normalized: exponent 254, mantissa all 1  $\Rightarrow (2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$ . The range is roughly  $10^{-38}$  to  $10^{38}$  — but precision is only about 7 significant decimal digits.*

**Teaching note:** The range/precision asymmetry is the crux students must internalize before the arithmetic section: floating point buys a *huge range* at the cost of *limited precision* (24-bit mantissa  $\approx 7$  digits). If you only remember one number from this section, it is “about 7 digits.”

Then rounding and precision loss. State the hard truth plainly: most real numbers are not exactly representable (0.1 repeats in binary), and IEEE 754 rounds to the nearest representable value (ties to even). The consequence — floating-point arithmetic is approximate — then has a concrete example.

**Example 3.4.2** (Where precision is lost). *Add  $10.75 + 0.1$  in single precision. 0.1 is stored as an approximation ( $\approx 1.100110011 \dots_2 \times 2^{-4}$ , rounded to 24 bits). Adding a large and a small number pushes the small one’s low bits out of the 24-bit window:  $10.75 + 0.1 \approx 10.85$ , but  $1\,000\,000 + 0.1$  rounds to **1 000 000** — the 0.1 vanishes. Precision is lost at the least-significant digits, worst when adding numbers of very different magnitudes.*

**Teaching note:** The  $1\,000\,000 + 0.1 = 1\,000\,000$  example is the one that makes precision loss *real* for students — nobody is surprised that 0.1 is approximate, but everyone is surprised that it can vanish entirely. Lead with it, then use the smaller case for the mechanics.

Close with the float-equality Socratic check: computing  $x = 0.1$  ten times by addition does not guarantee  $x == 1.0$ , because each add accumulates rounding error. The rule: compare with a tolerance, never with `==`.

### 3.5 Floating-Point Arithmetic

Motivate with the alignment problem: to add  $1.5 \times 2^3 + 1.25 \times 2^5$ , the exponents differ, so the hardware must first **align** the smaller mantissa. This is exactly where the low-bit loss comes from.

**Definition (Floating-Point Add).** *Four steps (P&H, Sec. 3.5, p.205) [2]:*

1. **Align:** *shift the smaller-exponent mantissa right until exponents match.*
2. **Add:** *add the aligned mantissas.*
3. **Normalize:** *shift to a leading 1; adjust the exponent.*
4. **Round:** *round to the stored width (ties to even).*

Multiply is easier: multiply mantissas, add exponents (subtract the bias once), normalize, round — no alignment step.

**Example 3.5.1** (Adding  $10.75 + 0.1$  (qualitative)).  $10.75 = 1.01011_2 \times 2^3$ ;  $0.1 \approx 1.100110 \dots \times 2^{-4}$ . *Align: shift 0.1's mantissa right by  $3 - (-4) = 7$  places. Add mantissas, re-normalize, round to 24 bits. The result is about 10.85, but the alignment truncated low bits of 0.1 — the sum is slightly off. The error is invisible in the 7-digit window; precision loss is **silent**.*

**Teaching note:** Students expect the arithmetic section to be “harder” than the format section. Flip that expectation: addition is four steps of bookkeeping, and the interesting part is the *alignment* step, which they have already seen as the source of precision loss. Connect the two explicitly so the arithmetic feels like a payoff, not a new topic.

Close with the multiply Socratic check: product fields are  $S = S_1 \oplus S_2$ ,  $M = M_1 \times M_2$  (renormalized),  $E = E_1 + E_2 - \text{bias}$ ; no alignment needed, but the mantissa product is wider and needs normalization and rounding.

## 3.6 Synthesis and Summary

Close the thread: **10.75** encoded as 0x412C0000; adding 0.1 showed the alignment step truncating low bits — silent precision loss. The encoding used every field of the format, and the arithmetic used every step of the add algorithm.

Set up Week 4: we now know how numbers are represented (integers, fixed point, IEEE 754) and how the ALU computes on them, but bits and adders do not run a program by themselves. Week 4 builds the machine — registers, the stack, instruction formats, and the fetch–decode–execute cycle.

**Summary — quick reference:** *Floating point* =  $(-1)^S \times 1.M \times 2^{E-\text{bias}}$ : range from the exponent, precision from the mantissa. *Single precision*: 32 bits = 1 + 8 + 23, bias 127, hidden 1. *Special values*: zero/denormal/infinity/NaN from reserved exponent patterns. *Add*: align, add, normalize, round. *Multiply*: multiply mantissas, add exponents, normalize, round. *Precision*: about 7 digits; rounding is silent and approximate — never compare floats with ==.

## References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.
- [2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.